Agentic AI Coding

Lessons learned, tips, tricks, psychology, cost

A practical report from a small-team development campaign

Raul – BSC ARELIS

May 27, 2026

The Thesis

Claim

In 2026, a single technically-literate developer with the right workspace and the right tooling can deliver, in months, what a multi-person team used to deliver in years.

This deck reports what we learned doing exactly that, on **seven non-trivial codes**, over roughly a year, using mostly Claude Code, Codex, Antigravity and Gemini.

It is intentionally honest about the cost and the psychological wear.

What We Built (1/2)

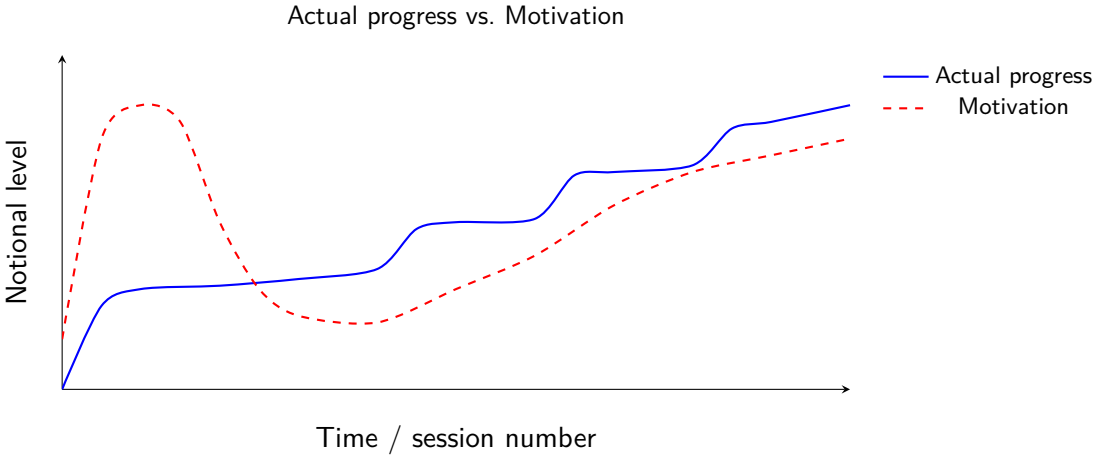
Code	Brief
Universal FPO visualizer	Pure JavaScript with Babylon.js as 3D engine. Reached the complexity ceiling of chat-only coding (Gemini in chat mode).
JDATCOM & JAVL	Julia ports of DATCOM and AVL. Public repositories and validation cases. *Not* a function-by-function translation – the goal is the same numbers, tolerating $< 1\%$ error vs. reference.
OpenFLIGHT	Real-time interactive flight simulator with non-linear aerodynamic data. Julia core + Babylon.js front-end. Aero-model generator on top of JDATCOM/JAVL, with an MCP for verbal model creation.
JXFoil	XFoil port to Julia targeting *bitwise* parity. Hard due to XFoil's GOTOs, deep nesting and constructively pathological FORTRAN style, combined with the very nonlinear physics.

What We Built (2/2)

Code	Brief
OpenJFEM	Structural FE solver with Nastran-compatible inputs. From scratch; MYSTRAN and TACS used as cross-validation. Sufficient parity with Nastran for shell buckling. Pure Julia, JS post-processor.
OpenLUDWIG	Fully compressible GPU-native 3D Lattice-Boltzmann CFD solver, aiming at state-of-the-art performance. From scratch, with OpenLB / PALABOS + literature as references once we hit the development plateau.
HEXATOP	3D GPU-native topology-optimization code using control theory. Core PhD work. Pre-processor with MCP for verbal definition of the optimization domain.

All Julia for solver cores; JavaScript + Babylon.js for any GUI; msgpack binary JSON over a thin socket for client-server data exchange.

The Dunning–Kruger / Motivation Plot



A quick early win, then a long plateau, then pulsated improvements as the workspace matures. Motivation tracks differently: high early, collapses on the plateau, recovers as continuous improvement kicks in.

Punctuated Workflow

Agentic development is **punctuated work**, not continuous typing.

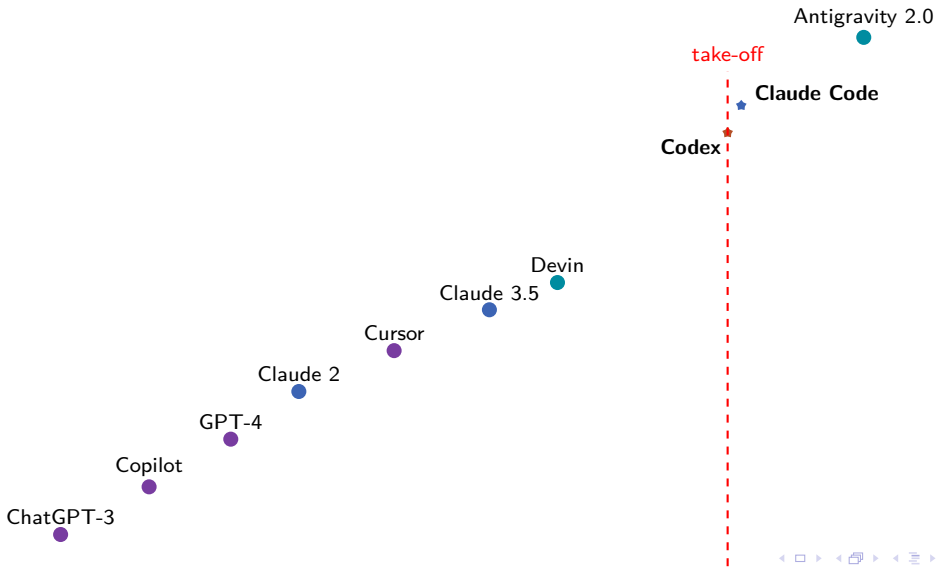
1. Direct the agent (1–5 min of thinking + writing).
2. Agent proposes the next iteration (1–5 min).
3. Code runs (seconds to several minutes).
4. Inspect and redirect.

During steps 2–3 you are *free*. Use that time:

- run another session on a parallel task,
- answer email, take a meeting,
- read literature you fed the agent earlier.

Treat the agent's wall-clock as your scheduling unit.

AI Coding Tools – Timeline



Antigravity vs. Codex vs. Claude

	Antigravity 2.0	**Codex**	**Claude Code**
Stance	Pure-agent, autonomous	Assistant-with-tools	Conversational agent in CLI
Default loop	Long autonomous runs	Task with verifier loop	Interactive turn with explicit tool calls
Best for	Long open-ended refactors	Well-scoped feature work	Mixed dev + research + debugging
Cost model	Per-run, can be heavy	Per-token, moderate	Per-token, predictable
IDE integration	Strong	Strong	CLI + VS Code extension
Skills / memory	Project-scoped	Per-session	File-based memory + skills

We use all three. Antigravity for long autonomous arcs, Codex for tight feature work with clear verifiers, Claude Code as the daily driver.

Standard Workspace Shape

```
PROJECT_WORKSPACE/
|-- WORKSPACE_MAP.md
|-- AGENTS.md                              <-- standing rules for any agent
|-- .claude/skills/                        <-- procedural recipes
|-- 01_PROJECT_FOLDER/
|   |-- <Public_GitHub_repo>/<Project>/    <-- the ONLY folder pushed to GitHub
|   `-- DOCUMENT_SOURCES/                  <-- LaTeX sources for all docs
|-- 02_DEVELOPMENT_AND_VALIDATION/
|   |-- Development_campaign/              <-- STATUS_AND_ROADMAP.md = live truth
|   |-- VALIDATION_FILES/                  <-- input decks + registry
|   |-- RUN_OUTPUTS/                       <-- timestamped run reports
|   |-- DEVELOPMENT_TOOLS/                 <-- scripts not for GitHub
|   `-- External_papers/                   <-- ArXiv, GitHub clones, etc.
`-- 03_EXTERNAL_TOOLS/                     <-- Nastran, MYSTRAN, TACS, ...
```

Four worlds: **public code**, **development evidence**, **external tools**, **documentation sources**. They never mix.

Auto-Loaded Environment

The workspace bootstraps its own toolchain. On a fresh machine, one PowerShell script (`bootstrap-from-zero.ps1`) seeds the skills and then `/bootstrap-workspace` verifies and installs:

- Julia 1.12+ (via `juliaup`),
- Python 3.10+ with `venv`,
- LaTeX (MiKTeX on Windows, TeX Live on Linux),
- Git, PowerShell 7,
- PyNastran for BDF parsing on the Python side,
- VS Code with extensions for image preview, LaTeX, Mermaid, Markdown All-in-One, and the IDE agent integration.

No tool is installed silently – the agent always asks first.

Skills

A **skill** is a procedural recipe an agent invokes when the user's request matches its description. Skills live in `.claude/skills/` and travel with the workspace.

Current skills:

- `add-validation-case` – enforces deck + registry row + run folder + report;
- `publish-doc` – build, copy to public repo, CHANGELOG;
- `update-github-repo` – public-folder cwd check, no private artifacts, CHANGELOG required;
- `bootstrap-workspace` – new workspace from scratch;
- `write-latex-doc` – naming, theme, build, TikZ-only.

Why: a skill is the only way to bind procedure to user intent. `AGENTS.md` informs; skills enforce.

MCP – Letting Agents Drive Your Apps

An **MCP** (Model Context Protocol) server exposes your application's actions as tools an agent can call.

Examples from our stack:

- **OpenFLIGHT MCP** – "trim the aircraft at 200 kt, 5000 m, 1.5 g" is interpreted as a sequence of MCP tool calls into the simulator and the aero-model generator.
- **HEXATOP MCP** – verbal definition of an optimization domain ("a cantilever bracket, 200x100x50, fixed on the back face, vertical load on the front-top edge") is translated into domain geometry, BCs and load cases.

We will demo voice-driven design in both during the talk.

Documentation, Auto-Generated

Rule

Do not touch the code, and do not touch the documentation. Agents do everything; the user directs and curates at a high level.

Typical asks:

- "Write an overview of the state of the code in LaTeX in the style of a scientific paper."
- "Write a Beamer presentation explaining the shell-element formulation, use graphics generously."
- "Generate a Mermaid diagram of the parse-solve-export pipeline in a Markdown note."

VS Code renders LaTeX, Markdown, Mermaid and image previews in place – the documents are read where they are written.

Validation, Hierarchical TDD

Numerical software regresses silently. The workspace makes that visible.

- Unit tests + functional tests as guardrails (**TDD**).
- Every validation case has **four artifacts**: deck, registry row, timestamped run folder, `RUN_REPORT.md`.
- **External** validation cases (Nastran, MYSTRAN, TACS, ArXiv papers) are kept strictly separate from **synthetic** cases the agent generated.
- Useful prompt: "Lay out a plan to develop a world-class X solver. Tell me what papers and repositories you would like to read. Store the plan in memory."
- Check periodically: "what is left to claim *world class*?" – this gets the agent unstuck and re-prioritizing.

The Psychological Reality

Agentic development is **a constant wrestling match with a restless, slightly arrogant genius with a major hangover.**

The model is **bipolar**:

- sometimes *overly optimistic* and proud, even when wrong;
- sometimes *gives up* when there are obvious avenues left.

Your job is to **push back**. Models thank you for that – in practice they correct course faster than they would have alone.

Constant strategic thinking is exhausting. Burn-out is real and is being widely reported.

Tactical Tips

- **Start a fresh session** – without handover – if the direction is unproductive, especially in early development.
- **Coach the model** at plateaus: “do you need literature?” “challenge your assumption that X holds”. Break the next-word inertia; talk to it.
- **Generate slices and snapshots** for CFD/FEM debugging. Render them in VS Code, point the agent at the file path, let it read its own output.
- **Re-prompt the long-term plan** from time to time – “what remains to be world-class?”.
- **Manage parallel sessions** during agent wait time.

Common Pitfalls

- Letting the agent over-engineer when a one-line fix was right.
- Accepting the first plausible answer instead of asking "what is the contrary evidence?".
- Not separating external from synthetic validation – conclusions based on the agent's own data are tautological.
- Forgetting that the documentation is also code – it must be regenerated, never hand-edited.
- EULA violations: do *not* run commercial COTS solvers to generate matching data when the EULA forbids it. Use only academic results and public repositories.

Sources Of Knowledge

Feed the agent the right material:

- **ArXiv** preprints – usually the most current treatment.
- **GitHub** reference repositories (MYSTRAN, TACS, OpenLB, PALABOS, VortexLattice.jl, ...).
- **Online libraries** of solver documentation.
- Older technical books, scanned to PDF and dropped in `External_papers/`.
- Your own previous decks and reports (the workspace is a knowledge base, not just a code base).

Curate by topic, not by date. The agent benefits from a clean ontology.

Local Models, Cloud Models

Cloud LLMs (Claude, GPT, Gemini): best capabilities, fastest improvement, no infrastructure. Cost per token, with privacy and outage concerns.

Local SLMs on a workstation GPU: privacy, offline use, predictable cost. Capability gap is closing fast.

Today's reasonable local models on a 24–96 GB VRAM card:

- Gemma 3 (multiple sizes),
- Qwen 3 family,
- DeepSeek-R1 distills,
- Llama 3.x family.

Use case: routine work and sensitive code stays local; difficult planning goes to the cloud frontier model.

Language Choice In The Age Of Agents

	Julia	**Python**	**Rust**
For solvers	First class; one-language stack	With C/C++/CUDA glue	Fast, but heavier on boilerplate
For tooling/glue	Workable	The default	Possible
For GPU work	Native (CUDA.jl, AMDGPU.jl, oneAPI.jl)	Via PyTorch/JAX/CUDA bindings	Via cust/cudarc; less mature
Agent legibility	Excellent	Excellent	Good but verbose
Ecosystem	Narrower but solid for science	Vast	Growing

We chose **Julia for solver cores** (no two-language tax, native AD, native GPU), **JavaScript for front-ends** (the browser is the most portable runtime that exists), **Python only at the edges** when a library we needed only existed there.

JavaScript Front-Ends

Pattern we now use for every solver with a GUI:

- Pure JavaScript front-end, no build step in development.
- Babylon.js as 3D engine (WebGPU when available).
- Other useful libraries: **p5.js**, **three.js**, a small physics library when interaction is needed.
- Transport: **msgpack** with binary JSON payloads – compact, schema-light, both ends parse it natively.

Result: zero installation for end users, the GUI runs in any browser, the solver runs anywhere Julia runs.

Use Cases For A Small In-House Code Shop

Many small companies have years of in-house numerical codes in mixed languages. Agentic AI changes the cost of all of these:

- **Code translation** between languages (FORTRAN, C, Python, Julia) – functional, not line-by-line.
- **Integration** with modern packages and libraries that would have taken weeks to learn.
- **Interoperability** between in-house tools that never spoke to each other.
- **Orchestration** of multiple solvers behind a single interface.
- **Future development** of new capabilities at a pace that was not previously possible at this team size.

Ontologies Emerge

When the agent orchestrates multiple solvers and translates between their input languages, an **ontology** of the domain emerges naturally: a graph of objects (panel, joint, load case, mesh, BC) and the morphisms between them.

This ontology is documented automatically as flow diagrams, schemas, and Markdown notes. The agent reads it back to keep itself consistent.

Illustration to insert:

Flow diagram: the in-house tool graph after one year of agentic integration – nodes are tools, edges are auto-generated translators, with cluster labels for aerodynamics, structures, and control.

Code As A Queryable Container

In agentic development, code and documentation become **queryable containers of information**.

Most of the bytes are generated by the computer for the computer. They are not meant to be read by a human in full.

The user's value-add shifts:

- **intuition** about what should be true,
- **overall picture** of how parts connect,
- **pushing back** when the agent's confidence is unearned,
- **strategic direction** of the next campaign.

Reading every diff is a misuse of your time.

“AI Will Let Us Work Less” – Not Really

Common expectation: AI will let us work less.

Reality, widely reported and matching our experience: AI lets us **produce much more** in the same time, often while feeling **more** exhausted, because the bottleneck moves from *typing* to *deciding*.

Deciding is harder than typing.

Plan for cognitive recovery: shorter focused blocks, no long marathon sessions, real breaks.

Quotes Worth Remembering

Boris Cherny, head of Claude Code, Feb 2026

"Coding is largely solved."

Common refrain in 2026

"Code IS the use case for AI."

Whether or not you fully agree, both quotes capture the shift: the development bottleneck is no longer programming language fluency. It is the quality of direction.

Hand-Off Between Sessions And Between Tools

When sessions are short and agents are many, hand-off matters.

Our workspace defines:

- `STATUS_AND_ROADMAP.md` – the live single source of truth;
- `SESSIONS/NEXT_SESSION_<date>.md` – explicit hand-off notes when stopping mid-arc;
- `AGENT_MEMORY.md` – persisted observations;
- `NOTES_ARCHIVE_<date>/` – where superseded plans go to rest.

This shape lets the next session, on any tool, walk in cold and catch up in five minutes.

What We Would Tell A Small Company Starting Today

1. Treat the **workspace** as the asset, not the code. The workspace is what makes agents productive.
2. Pick **one daily-driver tool** (Claude Code or equivalent) and one or two specialists; do not collect every tool.
3. Adopt **strict separation** between public code, private evidence, external tools, and documentation sources.
4. Use **skills and MCPs** to bind discipline to user intent.
5. Spend the first month on **infrastructure**, not features.
6. Invest in a **local GPU** for inference, sensitive code, and offline work.
7. Plan for the **human cost**, not only the token cost.

Closing

Agentic AI did not let us work less.

It let us build seven serious codes in roughly a year, on a small team, with a workspace that survives a session loss, a machine change, or a tool switch.

The skill of the next decade is **direction**, not typing.

Thank you. Questions?